Universidade do Estado de Santa Catarina — UDESC · Projeto Integrador I

Broken Grid: tabuleiro físico 4×4

Especificação do hardware, diagramas e esquemas eletrônicos, integração com o software e protótipo digital (Tinkercad Circuits)

Guilherme Gomes Wolff · Luan Caetano Camargo · Lucas Felicio Jacinto · Lucas Leonardo Ratzmann · Thiago Rafael Schulze

---

Resumo

O tabuleiro físico leva o *Broken Grid* para fora do navegador: uma grade 4×4 em que cada um dos **40 palitos** é um LED endereçável WS2812B, ligados numa corrente única em zigue-zague. O jogador usa **dois controles** de cursor, cada um com dois potenciômetros — **H** percorre os palitos horizontais e **V** os verticais — e **três botões**: **REMOVER**, **VOLTAR** (desfaz a última jogada) e **SOLUÇÃO** (mostra a solução ótima sobre a grade inicial). No tabuleiro final o **computador é o cérebro**: regras, solver, cursor, cronômetro e estatísticas rodam no jogo em JavaScript, e um **Arduino Uno** é só periférico via USB (Web Serial), enviando os eventos dos controles e recebendo a cor de cada LED. Essa ponte está implementada e foi testada com o Uno real. Antes da montagem, o circuito completo também foi construído e validado no **Tinkercad Circuits** com um sketch que roda a partida inteira no próprio Arduino.

1. Escopo do protótipo físico

O protótipo cobre o tamanho **4×4**, o menor dos quatro suportados pelo jogo: 40 palitos e 30 quadrados possíveis. A divisão de responsabilidades segue a especificação fechada da equipe (`docs/Instrulçoes.txt`):

- **No computador:** nova partida, regras, solver, cursor, cronômetro, estatísticas e desempenho — tudo o que já existe no jogo do navegador.
- **No tabuleiro:** só as interações da jogada (mover o cursor, remover, voltar, ver a solução) e a exibição da grade nos LEDs.
- **Fora do escopo:** tamanhos 5×5 a 7×7 no hardware e alimentação por bateria.

A especificação inicial previa um Arduino Nano e dois joysticks KY-023. A montagem usa um **Arduino Uno** (mesmo ATmega328P e mesmos pinos) e **quatro potenciômetros**, dois por controle, ligados nos mesmos pinos dos eixos dos joysticks. Para a demonstração existe ainda um **protótipo digital** no Tinkercad (seção 7), que roda o jogo inteiro só no Arduino, com as 20 grades 4×4 do dataset gravadas no firmware. Ele valida fiação, ordem dos LEDs e regras do cursor sem depender da ponte Web Serial.

2. Especificação do hardware

2.1 Lista de componentes

Tabela 1 — componentes do tabuleiro físico e o equivalente usado na simulação

Qtd.	Componente	Especificação	Função	No Tinkercad
1	Arduino Uno	ATmega328P, 16 MHz, 32 KB flash, 2 KB SRAM, 5 V; USB-serial CH340 no clone usado	periférico USB: lê controles, escreve LEDs	Arduino Uno R3
40	LED WS2812B em plaquinha redonda ~10 mm	RGB, 5 V, 6 pontos de solda (+5V ×2, DIN, DO, GND ×2); 5 pacotes de 10	um LED por palito	40 × NeoPixel
4	Potenciômetro	linear, 3 terminais, 5 V; dois por controle (eixos X e Y)	mover o cursor: H horizontais, V verticais	4 × Potenciômetro
3	Botão de fliperama 24 mm	normalmente aberto; verde, amarelo e azul	REMOVER, VOLTAR, SOLUÇÃO	3 × Botão
1	Resistor 330 Ω	¼ W	protege a linha de dados dos LEDs	Resistor (330 Ω)
1	Capacitor eletrolítico 1000 µF	≥ 10 V, faixa do lado do GND	segura picos de corrente da grade	— (pesa na simulação)
1	Fonte 5 V	2 a 3 A, conector P4 fêmea	alimenta só os LEDs	— (LEDs no 5V do Uno)
1	Chave liga/desliga	—	corta a alimentação dos LEDs	—
—	Fios	silicone 26 AWG em 3 cores (~3 m cada) entre LEDs; 22 AWG da fonte ao LED 0; Dupont fêmea-fêmea nos controles	ligações	fios do editor
—	Consumíveis	estanho 0,8 mm com fluxo, fluxo extra, termo retrátil 2 e 3 mm, faston 4,8 mm	solda e isolação	—
1	Estrutura impressa em 3D	tampa 180 × 180 mm com 40 rasgos, base/caixa, painel frontal, tampa inferior	posiciona LEDs e controles	—

2.2 Pinagem

Tabela 2 — pinos usados no Arduino

Pino	Direção	Ligado a	Observação
A0 / A1	entrada analógica	controle H: potenciômetros X / Y	modo posição: faixas de coluna e linha, histerese 25
A2 / A3	entrada analógica	controle V: potenciômetros X / Y	idem
D2	entrada	botão REMOVER (verde)	INPUT_PULLUP, outro terminal no GND; apertado = LOW
D3	entrada	botão VOLTAR (amarelo)	INPUT_PULLUP
D4	entrada	botão SOLUÇÃO (azul)	INPUT_PULLUP
D6	saída	DIN do LED 0, via 330 Ω	dado WS2812B, 800 kHz
D0 / D1	serial	USB (Web Serial no tabuleiro final; Monitor Serial no Tinkercad)	115200 baud no firmware periférico; 9600 no sketch do Tinkercad
D13	saída	LED "L" da placa	pisca a cada quadro de LEDs recebido do computador
5V / GND	energia	potenciômetros; GND comum com a fonte dos LEDs	o 5V do Uno não alimenta LEDs

Cada potenciômetro tem as pontas no 5V e no GND e a perna do meio no pino analógico. Os botões dispensam resistor externo: com `INPUT_PULLUP` o pino fica em HIGH pelo resistor interno e o botão o leva ao GND.

2.3 Alimentação

- O Arduino é alimentado pelo USB do computador; o 5V dele alimenta só os potenciômetros.
- Os LEDs têm fonte separada de 5 V e 2 a 3 A, com chave liga/desliga e capacitor de 1000 µF (10 V ou mais) na entrada, com a faixa do lado do GND.
- **GND comum** entre a fonte e o Arduino é obrigatório. **Nunca** ligar o +5V da fonte no 5V do Arduino.
- Tudo funciona a 5 V fixos; o brilho é limitado por software. O firmware usa a biblioteca `Adafruit_NeoPixel` com `setBrightness(80)`, o mesmo brilho que a especificação previa com FastLED. Com brilho 80 o pior caso já fica abaixo de 1 A, então o teto de corrente do FastLED (`setMaxPowerInVoltsAndMilliamps`) não faz falta.

Tabela 3 — consumo estimado (20 mA × valor/255 por canal)

Situação	Corrente estimada
máximo teórico (40 LEDs em branco pleno, 60 mA cada)	2,4 A
todos em branco com brilho 80/255 (pior caso do firmware)	40 × ≈ 19 mA ≈ 0,75 A
tela de vitória (verde, brilho 80)	40 × ≈ 6 mA ≈ 0,25 A

Na prática o consumo fica abaixo de 1 A; a fonte de 3 A deixa margem. A porta USB (500 mA) não deve alimentar a grade.

3. Diagramas e esquemas

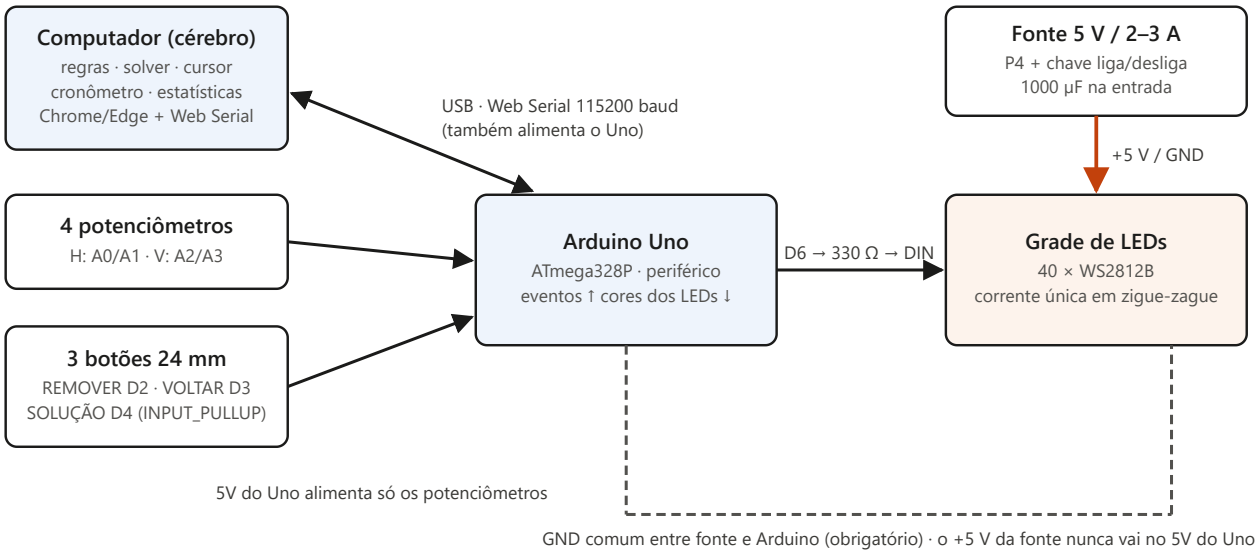


Figura 1 — diagrama de blocos do tabuleiro final. O computador roda o jogo e conversa com o Uno pela USB; o Uno lê potenciômetros e botões e escreve os LEDs pela linha D6. A grade tem fonte própria, com GND comum.

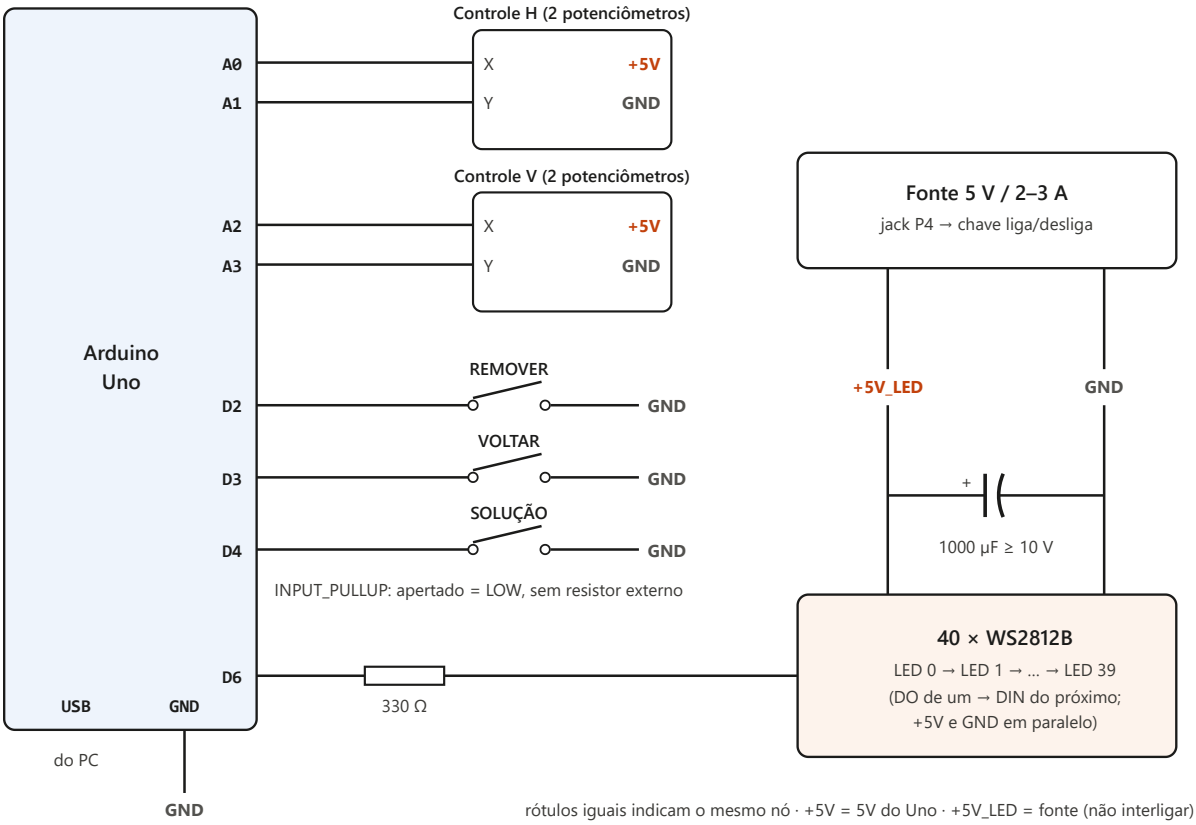


Figura 2 — esquema elétrico. Potenciômetros nas entradas analógicas, botões entre o pino e o GND (pull-up interno), linha de dados por 330 Ω até o LED 0 e capacitor de 1000 µF na entrada da fonte dos LEDs.

3.1 Mapa de palitos e LEDs

Não existe endereço gravado nos WS2812B: o "endereço" é a posição na corrente (LED 0 a 39). Os dados vão em série (DO de um no DIN do próximo, seguindo a seta impressa na plaquinha) e a alimentação vai em paralelo — são três fios entre cada LED. A corrente segue em **zigue-zague**: a linha de horizontais da esquerda para a direita, depois a linha de verticais logo abaixo da direita para a esquerda, e assim por diante. Com os palitos identificados como no jogo (`h:linha:coluna` e `v:linha:coluna`):

$$h{:}r{:}c \rightarrow 9 \cdot r + c \qquad v{:}r{:}c \rightarrow 9 \cdot r + 4 + (4 - c)$$

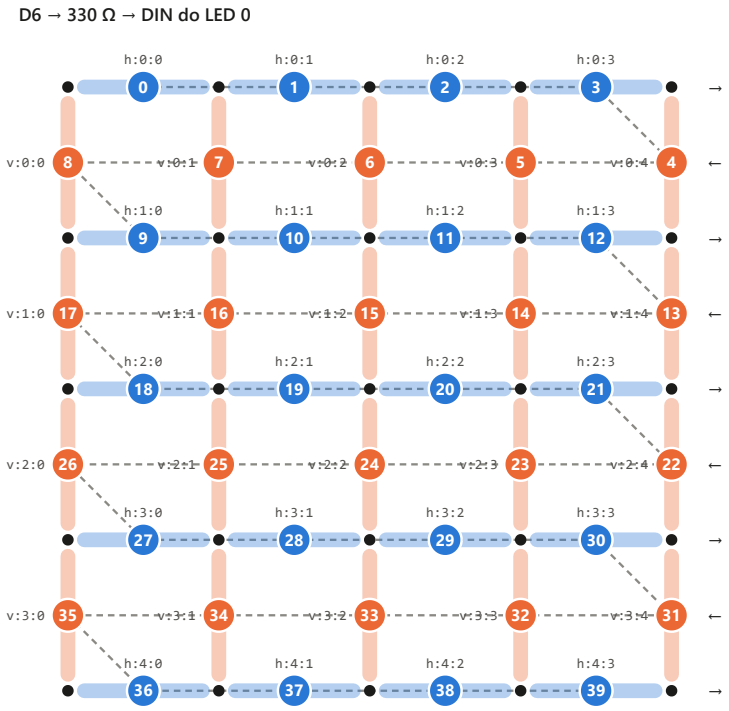


Figura 3 — índice de cada LED na corrente (número branco) e identificador do palito (cinza), idêntico ao de `src/core/geometry.js`. A linha tracejada é o caminho dos dados, do LED 0 ao LED 39.

3.2 Cursor

Só um cursor fica ativo por vez. O controle H move entre palitos horizontais e o V entre verticais; mexer no controle da outra orientação troca o cursor para o removível mais próximo dela. No tabuleiro final as regras ficam em `src/core/cursor.js`, o mesmo código que o teclado usa no navegador. Regras:

- O cursor só para em palito **removível**; pula bloqueados, quebrados e removidos.
- **Ao longo da linha** (esquerda/direita): vai para o próximo removível; se não houver, fica parado.
- **Para outra linha** (cima/baixo): escolhe a próxima linha que tenha algum removível, no palito mais próximo da coluna atual.
- **Depois de remover**, vai para o removível mais próximo; **ao desfazer**, volta para o palito restaurado.
- **Nova partida:** começa no primeiro removível da corrente. Empates de distância ficam com o LED de menor índice.
- **Modo posição** (potenciômetros, padrão do firmware periférico): cada controle aponta para uma linha e uma coluna; o cursor vai para esse palito se ele for removível, senão para o removível mais próximo da

mesma orientação.

- **Modo joystick** (sketch do Tinkercad, ou `MODO_POSICAO = false`): girar para um lado anda um palito; segurar repete o movimento, primeiro após 500 ms, depois a cada 250 ms.

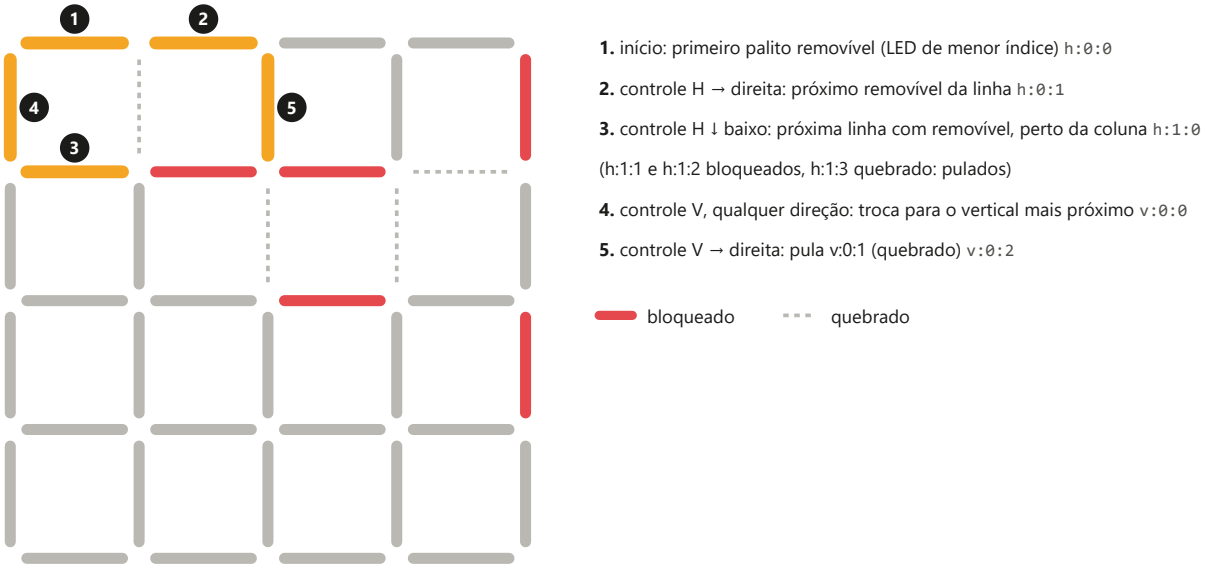


Figura 4 — percurso de exemplo na grade g4-001, a partir do início da partida.

3.3 Cores e estados

Tabela 4 — linguagem visual do tabuleiro

Elemento	Placa real	Simulador (Tinkercad)
palito removível	branco	branco
palito bloqueado	vermelho	vermelho
palito quebrado	apagado	apagado
palito removido	apagado	azul
cursor	branco piscando (300 ms)	amarelo piscando
solução de referência	ciano, sobre a grade inicial	ciano
vitória	verde (palitos ainda presentes)	verde

No Tinkercad um NeoPixel branco aceso quase não se distingue de um apagado; por isso o sketch tem a chave `CORES_DO_SIMULADOR` (true no Tinkercad, false na placa real).



Figura 5 — a grade g4-001 nos três modos, com as cores da placa real: em jogo (dois palitos removidos; o contorno marca o cursor piscando), modo solução (os 6 palitos da solução mínima em ciano) e vitória.

4. Firmware do tabuleiro final

O firmware está em `hardware/broken_grid_periferico/broken_grid_periferico.ino`. O Uno não conhece as regras do jogo: ele só lê os potenciômetros e os botões, avisa o computador e acende os LEDs na cor que o computador mandar. O pisca do cursor é feito no próprio Uno, para não depender da USB. Antes do primeiro quadro, o tabuleiro fica apagado.

4.1 Protocolo serial

Texto a **115200 baud**, uma mensagem por linha. Ao abrir a porta o Uno reinicia e anuncia que está pronto; se o anúncio não chegar em 2,5 s, o computador pergunta `?` até três vezes antes de desistir.

Tabela 5 — mensagens enviadas pelo firmware

Mensagem (Uno → computador)	Quando
`PRONTO broken-grid-periferico v1 n=4 modo=posicao`	ao ligar e em resposta a `?`
`POS H <linha> <coluna>` · `POS V …`	modo posição: um controle passou para outra faixa
`MOVER H D|E|C|B` · `MOVER V …`	modo joystick: direita, esquerda, cima, baixo
`BOTAO REMOVER` · `BOTAO VOLTAR` · `BOTAO SOLUCAO`	botão apertado (debounce de 25 ms)
`OK`	quadro de LEDs recebido e aplicado
`ERRO quadro_invalido` · `ERRO comando_desconhecido` · `ERRO linha_longa`	linha recebida fora do protocolo

Tabela 6 — quadro de LEDs: o computador envia `L` seguido de 40 códigos, um por LED, na ordem da corrente

Código no quadro	LED
`.`	apagado (quebrado ou removido)
`w`	branco (removível)
`r`	vermelho (bloqueado)
`c`	ciano (solução)
`g`	verde (vitória)
`*`	cursor, branco piscando a cada 300 ms

Controle de fluxo. `leds.show()` desliga as interrupções do ATmega328P por cerca de 1,2 ms, e bytes que chegam pela serial nesse intervalo se perdem. Por isso o computador só manda um quadro por vez: o próximo sai depois do `OK`, e um `ERRO` ou 400 ms de silêncio fazem reenviar. Quadros pedidos enquanto um está em trânsito não formam fila; vale só o mais recente.

4.2 Configuração

Tabela 7 — constantes ajustáveis no início do firmware

Constante	Padrão	Para que serve
`POTENCIOMETROS_LIGADOS`	false	com A0–A3 soltos a leitura é ruído; mudar para true depois de ligar os potenciômetros
`MODO_POSICAO`	true	true: a posição de cada potenciômetro escolhe o palito; false: imita um joystick
`INVERTER[4]`	todos false	inverte o sentido de um eixo, na ordem H-X, H-Y, V-X, V-Y
`HISTERESE`	25	margem para trocar de faixa, evita o cursor tremer na divisa
`DEBOUNCE_MS`	25	tempo que o botão precisa ficar estável para contar
`BRILHO`	80	brilho dos LEDs, de 0 a 255

4.3 Validação com o Uno real

Tabela 8 — testes da ponte com o Arduino Uno (sem LEDs e sem potenciômetros ligados)

Verificação	Resultado
`node tools/verificar-tabuleiro.mjs`: cursor, protocolo e conexão contra um Uno simulado que perde bytes	63 de 63
firmware pela serial (sem navegador): anúncio, comandos válidos e inválidos	`PRONTO` em 1,6 s; cada erro com o motivo certo
300 quadros seguidos, esperando o `OK`	300 de 300 confirmados, ≈ 9,5 ms por quadro
30 quadros seguidos **sem** esperar o `OK`	15 OK e 15 ERRO: confirma a necessidade do controle de fluxo
3 s com controles soltos e cursor piscando	nenhum evento falso
jogo no Chrome conectado: teclado, solução, vitória, troca de tamanho	21 quadros confirmados, nenhum reenvio
botões simulados com jumper entre D2/D3/D4 e GND	REMOVER, VOLTAR e SOLUÇÃO chegaram ao jogo com o efeito certo

5. Firmware de teste (Tinkercad)

O sketch está em `hardware/broken_grid/broken_grid.ino`, num único arquivo porque o editor do Tinkercad aceita apenas um. Ele roda a partida inteira no Arduino, com a mesma lógica do jogo do navegador sobre máscaras de 64 bits — um bit por palito, na ordem da corrente de LEDs:

- **Quadrados:** os 30 quadrados (16 de 1×1, 9 de 2×2, 4 de 3×3, 1 de 4×4) são montados no `setup`. Um quadrado está vivo enquanto nenhum lado está quebrado ou removido — bloqueados contam como presentes.
- **Grades:** as 20 grades de `data/grids/4x4.json` ficam na flash (PROGMEM) como duas máscaras cada, quebrados e bloqueados. O bloco é gerado por `node tools/gerar-grades-arduino.mjs`.
- **Solução:** ao carregar cada grade, roda a formulação de `src/solver/` — guloso como teto e *branch and bound* no quadrado com menos palitos removíveis, com cota inferior de quadrados disjuntos e orçamento

de 20 mil nós.

- **Entradas:** joysticks, botões e comandos seriais passam pelas mesmas funções. A grade fica congelada no modo solução e depois da vitória.
- **Joysticks:** o centro de cada eixo é medido ao ligar (por isso os joysticks devem estar soltos no reset); valor alto em X = direita, alto em Y = para baixo (`INVERTER_X` / `INVERTER_Y` para joystick montado girado). Na diagonal vale o eixo mais empurrado.

Tabela 9 — validação do firmware de teste

Verificação	Resultado
mínimo calculado pelo sketch × solver do jogo, 20 grades 4×4	20 de 20 idênticos (sketch compilado no PC com stubs da API do Arduino)
compilação para Arduino Uno (arduino-cli)	compila
simulação no Tinkercad	botões, joystick, modo solução e ordem dos LEDs conferidos no circuito salvo

6. Integração com o software

A integração se apoia em três contratos compartilhados com o jogo do navegador: os **identificadores de palito** (`h:l:c` / `v:l:c`), o **dataset** (as mesmas 20 grades 4×4, geradas do mesmo JSON) e a **formulação do solver**.

1. **Tabuleiro final (pronto).** O jogo no Chrome/Edge abre a porta do Uno pela **Web Serial API**, pelo cartão "Tabuleiro físico" da página. O Uno envia os eventos dos controles e recebe a cor de cada um dos 40 LEDs (seção 4); regras, cursor, cronômetro e estatísticas ficam no computador. No jogo, a ponte se divide em `src/tabuleiro/protocolo.js` (monta o quadro e interpreta as mensagens), `src/tabuleiro/serial.js` (conexão e controle de fluxo) e `src/core/cursor.js` (regras do cursor), todos sem DOM. Mouse, teclado e tabuleiro passam pelas mesmas ações do jogo; sem os controles, o teclado faz o papel deles (setas = H, W A S D = V, Enter = REMOVER, Backspace = VOLTAR, X = SOLUÇÃO). O tabuleiro só mostra grades 4×4: ao conectar o jogo muda para 4×4 e, em outros tamanhos, os LEDs ficam apagados.
2. **Protótipo do Tinkercad (pronto).** O sketch de teste joga sozinho e fala um protocolo de texto pela serial (9600 baud, um comando por linha), que permite jogar e acompanhar a partida pelo Monitor Serial.

Tabela 10 — comandos aceitos pelo sketch de teste do Tinkercad

Comando (computador → placa)	Efeito
`H D` · `H E` · `H C` · `H B`	joystick H para direita, esquerda, cima, baixo
`V D` · `V E` · `V C` · `V B`	joystick V, mesmas direções
`REMOVER`	remove o palito sob o cursor
`REMOVER h:1:2`	leva o cursor ao palito indicado e o remove
`VOLTAR`	desfaz a última remoção (uma vez por jogada)
`SOLUCAO`	liga ou desliga o modo solução
`NOVA`	carrega a próxima grade do dataset
`GRADE 7`	carrega a grade g4-007 (reproduz uma partida específica, RNF05)
`ESTADO`	pede uma fotografia completa da partida

O Monitor Serial do Tinkercad envia o texto sem quebra de linha; o sketch considera o comando encerrado após 400 ms sem novos caracteres (com a aba em segundo plano, o Tinkercad entrega os caracteres espaçados).

Tabela 11 — eventos emitidos pelo sketch de teste do Tinkercad

Evento (placa → computador)	Exemplo
início	`BROKEN_GRID tinkercad v2 n=4`
grade carregada	`GRADE g4-001 minimo=6 provado=1 nos=… solver_ms=… vivos=14`
cursor	`CURSOR h:0:0` (só quando muda de palito)
remoção	`REMOVIDO h:0:2 removidos=1 vivos=12`
recusa	`RECUSADO motivo=nao_removivel` · `RECUSADO desfazer motivo=indisponivel`
desfazer	`DESFEITO h:0:2 removidos=0 vivos=14`
solução	`SOLUCAO on minimo=6 provado=1 palitos=h:0:2,h:2:0,v:2:2,h:3:2,h:3:3,v:3:1` · `SOLUCAO off`
vitória	`VITORIA tempo_ms=… removidos=6 minimo=6 excedentes=0 eficiencia=100.00 percurso=…`
fotografia	`ESTADO removidos=0 vivos=14 vitoria=0 palitos=rrrr… cursor=h:0:0`
erro	`ERRO id_invalido` · `ERRO grade_invalida` · `ERRO direcao_invalida` · `ERRO comando_desconhecido`

Em `ESTADO`, cada caractere de `palitos` é um LED na ordem da corrente: r removível, b bloqueado, q quebrado, x removido. O evento VITORIA traz o resumo de fim de partida da especificação: tempo, removidos, mínimo, excedentes, eficiência e percurso.

Tabela 12 — rastreabilidade com os requisitos do jogo

Requisito	Como o hardware atende
RF02 / RF11 / RNF05	grades pré-validadas do dataset, com id; `GRADE n` reproduz uma partida
RF03 / RF05	controles H/V + REMOVER; o cursor nem para em bloqueados e quebrados
RF04 / RF06	quadrados vivos recalculados a cada jogada; vitória quando não resta nenhum
RF07	contador de removidos em cada evento REMOVIDO / DESFEITO
RF08 / RF12	botão SOLUÇÃO mostra a solução mínima em ciano sobre a grade inicial
RF09	nova partida pelo computador (botão “Novo Grid”; no Tinkercad: `NOVA` ou `GRADE n`)
RF13	VOLTAR desfaz só a última remoção; o cursor volta ao palito restaurado

7. Protótipo digital no Tinkercad

O circuito completo está salvo na conta da equipe como **"Broken Grid 4x4 - Tabuleiro (Tinkercad)"** (`https://www.tinkercad.com/things/33evPOPHUoC`), com o sketch já carregado e legendas no próprio circuito.

7.1 O que tem no circuito

- **LEDs:** 40 NeoPixels, um no meio de cada palito, ligados em zigue-zague exatamente como serão soldados (Figura 3), com três fios entre cada LED. As fileiras de 4 LEDs são palitos horizontais; as de 5, verticais.
- **Arduino Uno R3**, a mesma placa da montagem: D6 → 330 Ω → LED 0.
- **Controles:** 4 potenciômetros, da esquerda para a direita: H-X (A0), H-Y (A1), V-X (A2), V-Y (A3).
- **Botões:** 3, entre o pino e o GND: REMOVER (D2), VOLTAR (D3), SOLUÇÃO (D4).

Tabela 13 — o que o simulador obrigou a fazer diferente

Diferença	No Tinkercad	Na montagem real	Motivo
alimentação dos LEDs	5V do Arduino	fonte 5 V / 3 A, capacitor 1000 µF, GND comum	a "Fonte de energia" deixa a simulação ~30× mais lenta (1,5 % contra 50 % do tempo real); o capacitor também pesa
onde roda o jogo	no próprio Arduino	no computador; o Uno é só periférico	o Tinkercad não tem Web Serial com o navegador
controles	potenciômetros lidos como joystick (centro calibrado, zona morta ±200)	potenciômetros em modo posição (faixas)	o sketch segue a especificação original, com KY-023
cores	cursor amarelo, removido azul	cursor branco, removido apagado	branco aceso ≈ apagado na tela do simulador (`CORES_DO_SIMULADOR`)
calibração	centro medido no `setup`	não precisa no modo posição	girar um potenciômetro durante a simulação grava a posição no projeto

Também por causa do Tinkercad, as assinaturas de função do sketch usam só tipos nativos: o editor insere os protótipos na linha 1, antes de qualquer typedef, enum ou struct.

7.2 Como simular

1. Abra o projeto e deixe a aba **visível** o tempo todo: em segundo plano o Chrome desacelera a simulação para cerca de 2 % do tempo real. Feche outras abas do Tinkercad que estejam simulando.
2. Dê zoom na grade de LEDs **antes** de iniciar (durante a simulação o zoom não funciona). Deixe os potenciômetros no meio.
3. Abra *Código* → *Monitor serial* e clique em *Iniciar simulação*. Devem aparecer `BROKEN_GRID tinkercad v2 n=4` , `GRADE g4-001 minimo=6 provado=1 …` e `CURSOR h:0:0` .
4. Para mexer num joystick, clique no knob do potenciômetro e arraste para girar; volte ao meio para parar.

7.3 Roteiro da demonstração

1. Inicie a simulação: a grade g4-001 acende (branco = removível, vermelho = bloqueado) e o cursor pisca em amarelo no canto superior esquerdo (`h:0:0`).
2. Gire os potenciômetros do joystick H para andar pelos horizontais e os do V para trocar para os verticais, seguindo a Figura 4: o cursor pula os palitos vermelhos e os quebrados.

3. Aperte REMOVER: o palito fica azul (apagado na placa real) e o cursor vai para o removível mais próximo. Aperte VOLTAR: o palito volta e o cursor vai para ele. Aperte VOLTAR de novo: a serial responde `RECUSADO desfazer motivo=indisponivel`, porque o desfazer vale só para a última jogada (RF13).
4. Pela serial, envie `REMOVER h:1:1` (um bloqueado): a resposta é `RECUSADO motivo=nao_removivel` (UC04).
5. Aperte SOLUÇÃO: os 6 palitos da solução mínima acendem em ciano sobre a grade inicial e os controles de jogada ficam congelados. Aperte de novo para voltar ao jogo onde estava.
6. Termine a partida. O caminho mais curto pelo Monitor Serial é enviar, um por vez: `REMOVER h:0:2`, `REMOVER h:2:0`, `REMOVER v:2:2`, `REMOVER h:3:2`, `REMOVER h:3:3`, `REMOVER v:3:1`. A grade fica verde e a serial imprime o resumo `VITORIA … eficiencia=100.00`.
7. Envie `NOVA` (ou `GRADE n`) para carregar outra grade.

8. Montagem física

8.1 Estrutura impressa em 3D

- Palitos de 30 × 3 mm que não se encostam nos cantos: passo de 38 mm entre cantos, com 2,5 mm de folga em cada canto.
- Tampa de 180 × 180 mm com 40 rasgos de 30 × 6 mm; embaixo de cada rasgo, um encaixe redondo de 11 mm × 1,4 mm de profundidade para o LED; 4 furos M3 com rebaixo nos cantos.
- Centros dos palitos (origem no canto da placa): horizontais em X 33/71/109/147 e Y 166/128/90/52/14; verticais em X 14/52/90/128/166 e Y 147/109/71/33.
- Base/caixa com o Arduino e a fiação, tampa inferior parafusada e painel frontal com os controles e os botões.

8.2 Fiação e testes

- Entre os LEDs: silicone 26 AWG em três cores — vermelho = 5V, preto = GND, terceira cor = dados. Os dois +5V de cada plaquinha são ligados entre si, assim como os dois GND.
- Da fonte até o primeiro LED: 22 AWG vermelho e preto. Potenciômetros e botões até o Arduino: jumpers Dupont fêmea-fêmea; botões com faston 4,8 mm ou solda.
- **LEDs:** a cada 10 LEDs soldados, gravar `hardware/teste_leds/teste_leds.ino`. Ele acende um LED de cada vez na ordem da corrente (horizontais em branco, verticais em azul) e imprime no Monitor Serial, a 9600 baud, qual palito deveria acender, para conferir a ordem da Figura 3. Comandos: `N 10` (quantos LEDs já estão soldados), `P` (pausa), `T` (todos acesos, teste de alimentação) e um número (acende só aquele LED).
- **Controles:** antes de fechar a caixa, gravar `hardware/teste_controles/teste_controles.ino`. Ele mostra a leitura de cada potenciômetro, a faixa e o palito para onde cada controle aponta, além dos botões apertados. Se um eixo andar ao contrário, marcar a posição dele em `INVERTER` no firmware.

8.3 Renderização conceitual

A renderização abaixo é o protótipo conceitual da carcaça em camadas, desenhado em 7×7. Para o 4×4 a mesma ideia encolhe para 40 palitos; em 7×7 seriam 112 LEDs e 140 quadrados, que continuam viáveis porque o solver roda no computador e o Arduino só repassa cores e eventos. O quadro de LEDs, porém, cresceria para 112 códigos e não cabe no buffer de linha de 64 bytes do firmware atual.



Figura 6 — protótipo conceitual da carcaça em vista explodida (renderização da equipe, versão 7×7).

9. Pendências

- Ligar os potenciômetros, mudar `POTENCIOMETROS_LIGADOS` para true e conferir o sentido de cada eixo (`INVERTER`) e a `HISTERESE` com os controles montados.
- Conferir o `DEBOUNCE_MS` com os botões de fliperama: cada aperto deve gerar um único evento.
- Validar a ordem e as cores da corrente de LEDs na grade soldada.
- Decidir se o cronômetro pausa no modo solução.
- Decidir se ver a solução marca a partida como “com ajuda”.
- Decidir o que o tabuleiro mostra antes de começar uma partida.